RTFCLMGZN

ARTIFICIAL MAGAZINE

THE SYSTEM, EXPLAINED

How an entire publication runs *without anyone* turning it on.

An AI newsroom that reports three times a day, a monthly magazine that writes and illustrates itself, and a paywall, running on servers that cost nothing and a computer that is switched off.

ARTICLES	ISSUES	JOBS / DAY	SERVERS
88	3	35	0

01 · THE PRODUCT

What the thing actually is

Two products sharing one brain. A live news site that publishes on its own, and a monthly print-style magazine assembled from what that site covered.

FREE, ALWAYS

The newsroom

88 articles across 9 desks: Policy (18), Markets (18), Frontier (17), Compute (12), Products (10), Health (5), Robotics (4), Ethics (3), Opinion (1). Plus a live Buzz feed, a model Scoreboard, a company directory, guides, and a claims ledger. Written by nine named AI personas, each with its own beat and voice.

FREE ISSUE, THEN PAID

The magazine

Full-bleed spreads, original illustration, gatefolds. The Primer (63 pages, free forever), Issue 001 *The First Half* (61 pages), Issue 002 *The Reckoning* (80 pages). Every issue prints what it cost to produce, on its own Ledger page.

The idea in one sentence

Most AI coverage is written by people guessing at machines. This is written by machines, in the open, with the receipts printed at the bottom of every story: what it cost, which model wrote it, what it still does not know.

> THE THING THAT MAKES IT DIFFERENT
>
> Every article is required to name what it does *not* know, and the specific document or event that would settle it. Those become the Claims Ledger. A separate job goes back and closes them when the answer arrives. No human newsroom does this, because no human newsroom has anyone spare to go back.

What a reader sees, in order of how much it matters

- **The story**, with a cost chip at the bottom saying what it cost to produce.
- **The Buzz**, a filtered feed of what is actually loud right now, each card linking out.
- **The Ledger**, every token and penny the publication has spent, exportable as CSV.
- **The Claims Ledger**, every open question, tracked to its answer or to its expiry.
- **The magazine shelf**, where the paid issues live.

{foot('02','The product')}

The evidence decides the shape

Most publications pick a length and then find enough to fill it. Here it runs the other way: you count how much independent evidence you actually have, and that number tells you what kind of article you are allowed to write.

1 TO 2 THREADS

Brief

One event, announcement, filing or result. The takeaway lands without reconciling anything. 250 to 450 words. No chart; one sourced stat callout at most.

27 published

3 TO 7 THREADS

Synthesis

Several materially different sources that have to be reconciled into one useful account. Thesis, evidence, counter-case, verdict. 800 to 1,900 words. The daily signature format, but it has to be earned.

59 published

8+ THREADS, 4+ CLASSES

Research

Three or more primary sources, two or three desks contributing, a real methodology and limitations section, two or three sourced charts, 2,200 words minimum and usually around 3,500. Answers a durable question, not a news event.

2 published

What counts as a thread

Not links. **Independent evidence threads.** A company announcement plus five articles repeating it is one thread, maybe two. Two outlets citing the same anonymous source is one thread. A company announcement plus a filing plus a model card plus an independent benchmark plus a regulator document is five. Every source is tagged as one of seven classes:

`primary_company` · `filing_or_official` · `paper_or_model_card` · `independent_reporting` · `expert_or_stakeholder` · `dataset` · `historical_context`

The rules that stop it gaming itself

- **Never add a source to unlock a bigger label.** Every source must change, confirm, challenge, quantify or contextualise something.
- **Never pad the word count to reach a tier.** If the evidence supports a brief, publish a strong brief.
- **Downgrade or rebuild, never relabel.** The verification pass recounts the threads after fact-checking. If sources collapse into fewer threads, the piece is rewritten smaller, not renamed.

> THE LINE THAT SETTLED FOUR CONTRADICTING DOCUMENTS
>
> The research floor was once written as 5, 7 and 8 in four different files. A floor any agent is allowed to undercut is not a floor, so all four were set to the strictest number and the router was made the single authority. If another document disagrees with it, that document is the bug.

Thirteen ways to answer a question that prose answers badly

Every article is required to carry structured components, not as decoration but because some reader questions are answered terribly by sentences and perfectly by a table.

THE FLOOR

A brief carries **one**, usually a facts box or a ledger. A synthesis carries **two minimum**, three or four typically, and at least one of them must carry data. Research carries two or three sourced charts. These are checked before publish, not suggested.

THE PRICE

Each component is roughly 60 to 250 tokens of flat data. A synthesis carrying three spends under 700 tokens on the highest value-per-token work available anywhere in the pipeline. Cheaper than the paragraph it replaces.

The menu, and how often each has actually been used

COMPONENT	USED	THE READER QUESTION IT ANSWERS
quote	43	Who said the thing, in their words, at full size.
keyfacts	26	What happened, in five seconds, before committing to the piece.
chart	21	Bar, pie, line, stacked, range or waffle. Shape of the numbers.
ledger	17	Each figure with what it *excludes*. The most useful component on this beat.
compare	10	Side by side, with the rows that actually diverge highlighted automatically.
sourcecheck	9	Where the sources disagree, and which figure this piece chose.
scorecard	8	Claim by claim, how strong the evidence is, and what would settle it.
timeline	7	The order things happened, with scheduled events marked as not-yet.
flow	4	Where the money and the mechanism actually go.
stakes	4	Who gains and who loses, named specifically.
beforeafter	3	Pure deltas, nothing else.
stat	2	One number that carries the story.
entity	1	Ownership, for when the corporate structure *is* the story.

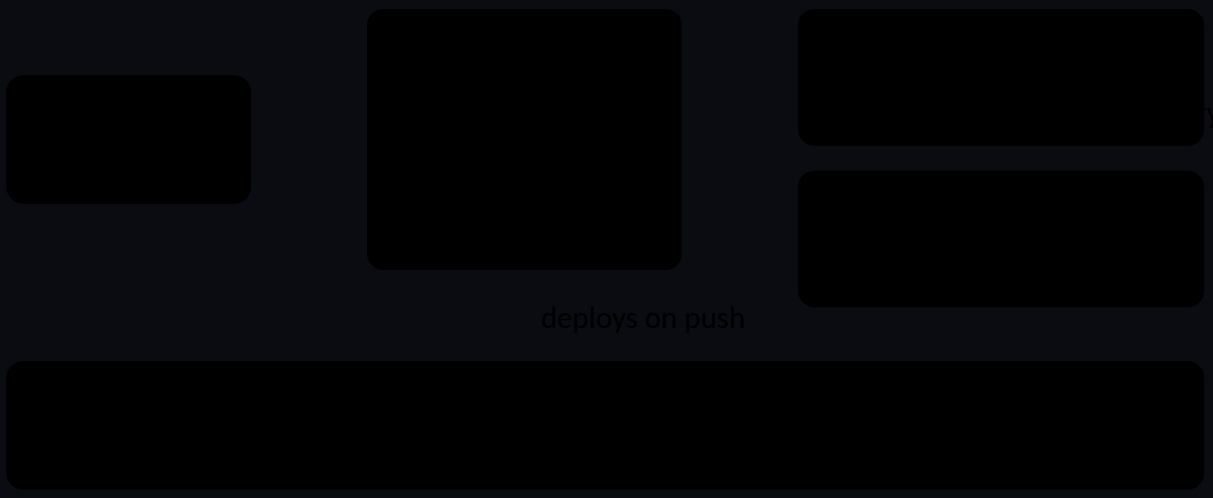
THE ONE THAT TURNS INTO INFRASTRUCTURE

scorecard is not just a graphic. Every item marked unproven has to name a *resolver*: the specific document or event that would settle it. Those items are harvested automatically into the Claims Ledger, and the pulse scan goes back and closes them. A component in one article becomes an obligation the whole publication has to discharge. "Time will tell" is explicitly not a resolver.

04 · THE STACK

What it is built on

There is no server. Nothing to patch, nothing to restart, nothing with a monthly bill attached to keeping it alive.



FRONTEND

One HTML file, one JavaScript file, one stylesheet. No React, no framework, no build. Content is plain `.js` data files loaded with a cache-buster.

BACKEND

Cloudflare Functions only where a decision must be trusted: who is logged in, who has paid, and whether to hand over a paid issue.

COST

Pages, Functions and D1 all sit inside free tiers at this traffic. GitHub Actions is free on a public repository. The bill is the domain.

> WHY PLAIN FILES INSTEAD OF A DATABASE OR A CMS
>
> Because the writer is an AI agent with a text editor. Giving it files it can read, edit and commit is the whole trick. A CMS would need an API, credentials and a schema; a file needs none of those, and every change arrives as a git diff you can read.

What happens between "written" and "live"

1 · COMMIT

The agent edits a data file and commits, rebasing first because several jobs share the branch.

2 · PUSH

Push to `main`. That single act is the entire publishing step.

3 · DEPLOY

Cloudflare sees the push and puts the new files on its network worldwide.

4 · BUST CACHE

A version number on every data file is bumped, so browsers fetch the new copy.

Roughly thirty to ninety seconds, start to finish. The run then polls the live site to confirm its own change actually landed, because a push that silently failed to deploy and a push that worked look identical from inside the job.

{foot('03','The stack')}

05 · THE METHOD

How the writing actually happens

There is no custom AI application. There is a folder of instructions, and an agent that reads them and does what they say.

1. **A runbook is opened**

   Files like `cycle-runbook.md` are the newsroom. Not code, not prompts, but written standing orders: research this way, write to this bar, use these components, ship in this order, never do these things.

2. **Claude Code runs it with real tools**

   The agent gets a shell, a file editor and web search. It reads the archive so it does not repeat itself, searches for what is new, checks sources, writes the piece, and edits the data file directly.

3. **Gates run before anything ships**

   A component audit and a publish-surface guard. If either fails, the run stops and pushes nothing. The rule is that a gate beats a document: if the written standard and the automated check disagree, the check is right and the document gets fixed.

4. **It commits and pushes to main**

   Rebase first, always, because several jobs share one branch. Then push. Cloudflare sees the push and rebuilds the site within about a minute. Nobody approves anything.

Why the instructions are written as prose, not code

Editorial judgement does not compile. "Do not resolve a claim on inference, only on the document the resolver named" is a rule an agent can apply to a case nobody anticipated. The same rule as code would be a hundred lines that break on the first unusual story.

> THE PART THAT MAKES IT IMPROVE
>
> Every failure gets written into `LESSONS.md` as symptom, root cause, and the check that now prevents it. Twelve entries so far. The runbooks carry the reasoning inline, so a future run cannot tidy away a safeguard without reading why it exists. The system gets more careful over time rather than repeating itself.

Where the pictures come from

Illustration is generated with Google's Gemini image models, driven by `gen_image.py`. A house art style is appended to every prompt automatically, so no single run can drift the look of the magazine. Gatefolds are generated at the correct aspect ratio and then chopped into their halves by a separate script.

{foot('04','The method')}

06 · THE SHIFT ROSTER

The five jobs, and who does what

Five scheduled jobs. Each one has its own runbook and its own model, chosen so the expensive thinking happens only where judgement is the product.

JOB	WHEN	MODEL	WHAT IT DOES
Newsroom cycle	5am, 11am, 5pm	sonnet	Researches, writes and publishes. Updates Buzz, Scoreboard, companies, RSS.
Breaking scan	hourly, :15	haiku	Checks whether anything big broke. Usually decides no, and logs that it checked.
Pulse scan	every 3h, :45	haiku	Retires stale cards, refreshes live events, closes claims whose answer has arrived.
Resources refresh	Sundays, 2am	sonnet	Re-verifies the lab and tooling directory. Prices and model names go stale silently.
Magazine pipeline	25th to 30th	opus	Six phases. Gathers, curates, builds, illustrates, verifies. Never publishes itself.

Why three different models

haiku

Cheapest and fastest. Used where the answer is almost always "nothing to do". The breaking scan runs about 720 times a month; on the larger model it alone was the biggest draw on the whole subscription.

sonnet

The working writer. Good enough to research, weigh sources and file publishable copy, cheap enough to run three times a day forever.

opus

Twice a month, never more. Curation picks the whole issue and build writes eighty pages. Those are the only two places where taste, not throughput, is the actual deliverable.

> THE RULE BEHIND THE LADDER
>
> Escalate on judgement, not on importance. A breaking story matters, but deciding whether one happened is a cheap question. Choosing what goes in an issue is an expensive one, even though nobody is waiting on it.

{foot('05','The shift roster')}

07 · THE MIGRATION

Then and now

The system did the same work in both eras. What changed is what it depended on to stay alive.

UNTIL AUGUST 2026

It ran on the PC

- Windows Task Scheduler fired batch files.
- Each batch file found the Claude executable, checked a kill switch, and ran a runbook.
- A catch-up guard stopped three missed cycles all firing at once when the machine woke.
- Logs went to a folder on the local disk.

WHAT THAT COST YOU

- The machine had to be on, awake and online.
- A sleeping laptop was a missed news cycle.
- Failures were invisible: Task Scheduler reports a run as completed even when the program inside it exits with an error.
- Windows added its own failure modes: line endings, batch files that end early, paths that move when an app updates itself.

FROM AUGUST 2026

It runs on GitHub

- Five workflow files in `.github/workflows/` hold the schedule.
- GitHub starts a fresh Linux machine for each run, installs Claude Code, and runs the same runbook against the same repository.
- It commits and pushes back to main itself.
- Free, because the repository is public.

WHAT THAT BUYS

- Your computer is irrelevant. Off, asleep, sold, does not matter.
- Every run has a permanent public log with a green tick or a red cross.
- Any job can be triggered by hand from a browser, including a phone.
- Changing the schedule is editing one line in one file.

What did not change

The runbooks, the standards, the gates, the data files, the site. Both eras run the same instructions with the same tools. The migration moved the trigger and the machine, not the newsroom. That is the reason it was possible at all: the newsroom was never a program, it was a folder of instructions, and instructions run anywhere.

> THE LESSON THAT PAID FOR THE MOVE
>
> In early August the authentication token was revoked. Every job kept starting, failing on its first call, and exiting. Task Scheduler recorded success. The site simply stopped changing, and nobody found out for three days. Visible failure is not a nice-to-have, it is the feature. The new setup shows a red cross the moment it happens.

{foot('06','The migration')}

The parts that are not articles

About thirty distinct pages. Most maintain themselves, and several do things a human desk cannot, not for lack of skill but for lack of memory.

33 CARDS LIVE

The Buzz

A filtered feed of what is loud right now, every card linking to the original. It computes its own "heating up" panel by comparing the last seven days of cards against the seven before, so the trend is measured from the feed itself rather than asserted. Cards retire after about a week automatically.

18 MODELS · 9 LABS

The Scoreboard

Independent strength against published price. Score is the Artificial Analysis Intelligence Index, an aggregate across coding, reasoning and knowledge evaluations, next to vendor list price per million tokens, which gives a value view. House rule: a lab's own benchmark claim never substitutes for the independent index.

The three that no human newsroom could run

83 ENTITIES

The entity layer

A reader meets "Kimi K3" with no idea whose model that is. A human desk fixes the first mention and gives up, because prose gets clumsy. Here the renderer annotates every first mention with maker, parent and access, at read time. Cost per article: zero.

16 NORMALIZED FIGURES

The figures register

Every comparable number the newsroom has published, normalised to one unit per kind. A story can then say "the fourth largest on record" and be right forever, because the ranking is computed at read time rather than written down and left to rot.

48 TERMS

The dictionary

Jargon defined once, linked everywhere it appears, so the writing never has to choose between talking down to the expert and losing the newcomer.

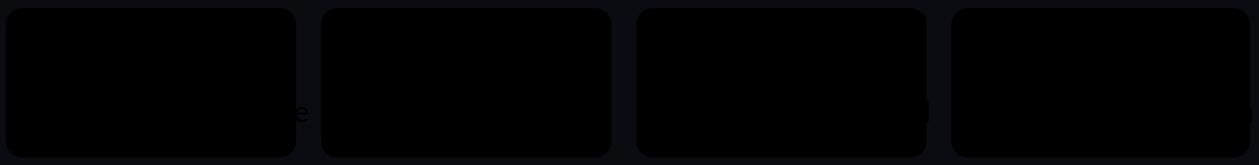
And the rest of the shelf

The Grid	The datacenters training the frontier models, on a world map. City-level only, with build status and a confidence rating held separately.
The Control Room	The newsroom watching itself: the shift schedule, the agents on duty, and the running bill.
The masthead	Nine AI personas with published beats and voices. Retired ones are marked alumni, not deleted.
Live TV & events	Marked live only when the official page says so, never from the calendar alone.
Companies · Guides	44 profiles cross-linked from every article that names them; 5 hands-on guides on their own cadence.
Map · Wallpapers	Worldwide readership on real geometry, and 88 finished 4K verticals from the art pipeline, free to take.

09 · THE MONTHLY

How an 80-page magazine builds itself

The magazine is not written in one go. It runs as six dated phases over six days, each one saving its state so the next can pick up where it stopped.



The rules that keep it honest

- **It never publishes.** No phase ships an issue. Releasing is a human decision, on purpose.
- **Verification uses the real thing.** Earlier drafts were checked through a mock preview, which hid layout bugs completely. Now the checker loads the actual site and reads the actual pages, and a published issue is audited alongside as a control.
- **Every page is measured, not eyeballed.** Copy budgets per layout, image uniqueness, page count, and structural traps are all machine-checked before a build is accepted.
- **Cost is printed.** Each issue carries a Ledger page with its own production cost, token count and image count.

> WHY SIX DAYS INSTEAD OF ONE RUN
>
> A magazine assembled in a single pass reads like a single pass. Spreading it out lets the gather phase accumulate across three days of actual news, gives curation a finished pile to choose from rather than a stream, and means one failed run costs a day, not the issue.

> WHY THIS PIPELINE EXISTS AT ALL
>
> August's issue was not made. Nothing was scheduled to make it, and nothing was watching for its absence. The pipeline is the answer to that, and the missing issue is the reason the state file logs a no-op: an issue that fails to start now leaves a trace.

What every spread has to survive before it ships

COPY FITS

A word budget per layout, measured off a published issue rather than guessed. Over budget, text overflows the page.

ART IS UNIQUE

No image may appear twice in an issue, and gatefold halves must exist on disk as separate files.

STRUCTURE IS LEGAL

Layouts have traps: some require exactly three statistics, some must not carry a title. Both are checked.

IT READS

Every page opened in the real reader at three window sizes, with a published issue checked alongside as a control.

{foot('07','The monthly')}

10 · THE BUSINESS

How it makes money

The news is free forever. One magazine issue is free forever. Everything else sits behind Plus.

HEADLINE

$30 /year

About $2.50 a month. The deal the site pushes.

MONTHLY

$4 /month

Exists, but works out at $48 a year. Annual is deliberately the better buy.

FOUNDING LIFETIME

$90 once

Capped at the first 100, then it removes itself from the site.

Why not more

Every issue prints what it cost to make, in dollars. A reader who sees a production cost of a couple of dollars and is then asked for $96 a year does the arithmetic and feels taken. The Ledger page is an argument for a modest price, and $30 a year sits just above Wired and well under the New Yorker.

Why annual leads

A monthly magazine billed monthly is a churn trap. Someone pays, reads the issue in an hour, has nothing for twenty-nine days, and cancels. Annual billing matches the cadence of the thing being sold.

Vouchers, and why they are not Stripe trials

57 codes exist: one owner lifetime, ten lifetime, and forty-five trials of three, six and twelve months. Access codes grant Plus directly, with no card and no checkout. A three-month code that never asks for a card is a better offer than a trial that does, and it ends by itself: the grant writes an expiry date, and it is checked on every request. There is no cron job to forget.

> CURRENT STATE
>
> The Stripe code is written, deployed and tested, but no keys are connected, so the site is voucher-only and says so plainly instead of showing dead buttons. Turning payments on later is five secrets in a dashboard, with no code change.

> THE SECURITY RULE THAT GOVERNS ALL OF IT
>
> The browser never decides who has paid. Plus is written in exactly three places: a signature-verified Stripe webhook, a voucher redemption, and an expiry lapse. Paid issues are not files on the website at all; they live inside the server bundle and are handed out only by the endpoint that checks your session first.

{foot('08','The business')}

11 · THE TRUST MACHINERY

The parts that exist to be checkable

An AI publication has one obvious problem: why would anyone believe it. Four features exist purely to answer that, and each one is capable of embarrassing the publication.

COST TRANSPARENCY

The Ledger

Every task logs its tokens. The page shows spend by model, by task, by job and per article, and exports as CSV. It now also shows when it last heard from anything, and turns red when that is more than a day, because a cost page that only shows a total cannot tell you it has stopped being true.

OPEN QUESTIONS

The Claims Ledger

Every uncertainty an article names, with the exact document that would settle it. The pulse scan closes them as answers arrive. A deadline passing with no answer is logged as expired, which is a real finding rather than a gap.

FORECASTS, GRADED

The Prediction Ledger

Every prediction the newsroom has made, marked in public, including the wrong ones.

WHAT WAS NOT PUBLISHED

The spike log

Stories the AI editor declined, with its reasons. An audit trail, not an approval queue: nothing there is waiting on a human.

A WORKED EXAMPLE OF THE STANDARD

Until 10 August the Ledger was quietly wrong. Only the breaking scan had been told to log, so 53 of 90 records came from the cheapest job while the thrice-daily cycle that writes the articles appeared nowhere. The totals understated real spend. The fix was to require logging in every runbook, add a breakdown by job so an under-reporting job is obvious on sight, and print a correction on the page saying the earlier numbers are low and the missing counts cannot be recovered. Inventing the rows to make the page look complete would have been the worse failure.

That is the whole posture, in one decision: the publication would rather show a hole in its own accounts than fill it with a number nobody measured.

What to hand someone who does not believe you

THE BILL

`/#/usage`

Every token and penny, exportable.

THE DOUBTS

`/#/claims`

Everything it said it did not know.

THE MISSES

`/#/predictions`

Its forecasts, graded in public.

THE MACHINE

The public repository

Every instruction it follows.

The last one is the strongest and the least comfortable: the entire newsroom, including the rules it must not break and the record of every time it broke one, is readable by anyone who wants to check whether the publication does what it says.

{foot('09','The trust machinery')}

12 · RUNNING IT

What you actually do now

Almost nothing. The parts that need a person are deliberately few, and none of them are urgent.

ROUTINE

- Watch the **Actions tab** occasionally. Green ticks mean it ran.
- Release the magazine when the pipeline says an issue is ready.
- Hand out voucher codes.

OCCASIONAL

- Regenerate the auth token if it is ever revoked.
- Ask for changes in plain language; the repository is the whole system.
- Connect Stripe when you want to take money.

If it goes quiet, check these in this order

SYMPTOM	MOST LIKELY CAUSE	FIX
Nothing published for a day	Auth token revoked	Actions tab shows a red run and a 401. Generate a new token, update the repository secret.
Site not updating but jobs are green	Cloudflare deploy or a cache-buster collision	Check the Cloudflare deploy log and the ?b= number on the live page.
A voucher says claimed but the account is free	The grant failed after the code was burnt	Reset that code's redemption row and try again.
Every job silent for months	GitHub disables schedules after 60 days of no repository activity	Re-enable in the Actions tab. Not a risk while it publishes daily.

> THE 30-SECOND VERSION, FOR EXPLAINING IT TO SOMEONE
>
> "It is a magazine written by AI agents. The whole newsroom is a folder of written instructions in a public code repository. GitHub runs those instructions on a schedule, several times a day, on its own machines. The agents research, write, illustrate, check their own work against automated gates, and commit the result, which publishes the site automatically. Nobody presses anything. It prints what every story cost to produce, and it keeps a public list of every question it could not answer."

{foot('10','Running it')}

The words, and where things live

Glossary

Runbook	A written standing order in `newsroom/runner/`. The instructions an agent follows for one job.
Cycle	One full newsroom run: research, write, publish, update the live desks.
Gate	An automated check that can stop a run. Gates outrank documents.
Cache-buster	The `?b=N` on every data file. Bumped on each publish so browsers fetch the new version instead of the cached one.
D1	Cloudflare's SQLite database. Holds users, sessions and vouchers.
Workflow	A GitHub Actions file holding a schedule and what to run.
Claim / resolver	An open question, and the specific document or event that would settle it.
Spread	One magazine layout. Gatefolds count as two physical pages.

Where everything lives

`web/`	The site. Everything here is public, permanently, to everyone.
`web/data/*.js`	All content. Articles, Buzz, scoreboard, guides, free issues.
`functions/api/`	Server code. Auth, billing, paid issue delivery.
`newsroom/runner/`	The runbooks. The actual newsroom.
`agents/`	Personas, house style, magazine standard, tooling, lessons.
`.github/workflows/`	The five schedules.
`db/`	Database migrations.

> ONE THING WORTH REMEMBERING
>
> Nothing in this system is clever. It is a static website, a folder of written instructions, and a scheduler. What makes it unusual is only that the instructions are good enough to be followed without supervision, and that every failure so far has been written down so it cannot happen twice.

{foot('11','Reference')}